Ejercicios Capítulo 6

Datos estructurados: Arreglos

1) A continuación se presenta un programa y su prueba de escritorio. Utilizarla como modelo en el caso de otras pruebas de escritorio con arreglos.

Lote de prueba: 4 , 3 , 0 , 2 , 1 , 10

```
PROGRAM EJEMPLO;
TYPE
    arr=array[1..10] of integer;
VAR
    i,n,j:integer;
    vect:arr;
Begin
    Writeln('Ingrese la cantidad de elementos');
    Readln(n);
    For i:=1 to n do
         readln(vect[i]);
    Writeln('ingrese un valor entero');
    Readln(j);
    For i:=1 to n do
         vect[i]:=vect[i]*j;
    Writeln('El vector modificado es');
    For i:=1 to n do
         Write(vect[i]:4);
end.
```

n	i	Vect	j	Salida
4				
	1	[3,…]		
	2	[3,0,...]		
	3	[3,0,2,...]		
	4	[3,0,2,1,...]		
			10	
	1	[30,0,2,1,...]		
	2	[30,0,2,1,...]		
	3	[30,0,20,1,...]		
	4	[30,0,20,10 , ...]		
				El vector modificado es
	1,2,3,4			30 0 20 10

Reemplazar en el programa EJEMPLO las líneas que se hallan en negrita por las que se dan a continuación y construir la prueba de escritorio:

Lote de prueba : 4 , 3 , 5 , 2 , 7 , 3

```
For i:=1 to n do
    vect[i+2]:=vect[i]*j;
Writeln('El vector modificado es');
For i:=1 to n+2 do
    Write(vect[i]:4);
```

2) Completar los procedimientos y el programa principal, de manera tal que resuelva lo siguiente: Leer tres vectores en las variable x, y, z, obtener todos los posibles vectores suma entre dos de ellos e imprimir por pantalla los resultados.

```
Program vectores;
 type
     vec=array[1..20] of integer;
 Var
     x,y,z,sxy,syz,sxz:vec;
     N:integer;

 Procedure Leevec(M:integer; var W:vec);
     ………………………………………………………………………………………………………………
     ………………………………………………………………………………………………………………
     ………………………………………………………………………………………………………………

 Procedure suma(M:integer;W,U:vec; var vsum:vec);
     ………………………………………………………………………………………………………………
     ………………………………………………………………………………………………………………
     ………………………………………………………………………………………………………………
 Procedure escribe(M:integer; W:vec);
     ………………………………………………………………………………………………………………
     ………………………………………………………………………………………………………………
     ………………………………………………………………………………………………………………
 Begin {pp}
     writeln('dimension de vectores?');
     readln(N);
     ………………………………………………………………………………………………………………
     ………………………………………………………………………………………………………………
     ………………………………………………………………………………………………………………
     ………………………………………………………………………………………………………………
     ………………………………………………………………………………………………………………
     ………………………………………………………………………………………………………………
     writeln('resultados');
     ………………………………………………………………………………………………………………
     ………………………………………………………………………………………………………………
     ………………………………………………………………………………………………………………
 end.
```

3) Escribir un programa que, a través de procedimientos y/o funciones, permita leer un arreglo B de N elementos enteros y luego presente un menú en el que el usuario pueda seleccionar una de las opciones siguientes:

a) Intercambiar los elementos de las posiciones h y k ($h \leq N$; $k \leq N$), elegidas por el usuario. Mostrar el arreglo modificado.
b) Asignar el valor cero a todos los elementos de índice impar. Mostrar el arreglo B modificado.
c) Calcular y mostrar la suma de todos los elementos del arreglo.
d) Construir un arreglo C que sea B permutado (C[1]=B[N], C[2]=B[N-1], …, C[N]=B[1]). Mostrar el arreglo C.
e) Imprimir los elementos del vector B que sean pares.

4) Desarrollar una función que a partir de un arreglo A de N elementos, cada uno de los cuales es una cifra entera, retorne el número cuyos dígitos son los elementos de A.

Ejemplo: [1 , 2 , 4 , 0 , 9] → 12409

5) Ingresar en un vector V un conjunto de números enteros, a continuación un valor entero K, y luego.

a) Mostrar la cantidad de ocurrencias de K en V, o el cartel "NO ESTA" en caso de que K no esté en V.
b) Generar y mostrar un arreglo W con los elementos de V distintos de K.
c) ¿Se podrían obtener, en un único recorrido, los resultados de los incisos a) y b)? En caso afirmativo, realizarlo.
d) Determinar e indicar si V es una *mochila perfecta*. Considerar que un vector es una *mochila perfecta* si cada elemento (a partir del tercero) es mayor que la suma de los dos anteriores.

6) Para vectores reales de R^3, defina funciones y/o procedimientos para calcular:

a) el modulo de un vector: $R^3 \rightarrow R$
b) un vector unitario con la misma dirección del vector dado: $R^3 \rightarrow R^3$
c) la suma de dos vectores: $R^3 \times R^3 \rightarrow R^3$
d) el producto escalar de dos vectores: $R^3 \times R^3 \rightarrow R$
e) el producto vectorial de dos vectores: $R^3 \times R^3 \rightarrow R^3$
f) el producto mixto, dados 3 vectores: $R^3 \times R^3 \times R^3 \rightarrow R$
g) dados dos vectores de R^3, ver si uno de ellos es combinación lineal del otro.
h) dados tres vectores de $R^3 QUOTE$, ver si alguno de ellos es combinación lineal de los otros dos (el producto mixto debe ser nulo).

7) Implementar un procedimiento para calcular el límite de un cociente de polinomios cuando $x \rightarrow \infty$

$$\lim_{x \rightarrow \infty} \frac{P(x)}{Q(x)}$$

Recibirá dos polinomios, cuyos grados no se conocen de antemano, como argumentos de entrada, y los argumentos de salida serán tres: el signo del límite (1 si el resultado es positivo, -1 si es negativo y 0 si es nulo), un valor A (acotado) o I (infinito), y en caso de ser acotado, el módulo del valor del límite. (Recordar o investigar cómo se resuelven los límites de funciones racionales cuando x tiende a infinito).

8) Una aplicación de registro de calorías quemadas durante el día utiliza la información del GPS del teléfono para determinar la velocidad a la que se desplaza el usuario y obtiene las calorías quemadas de acuerdo a la siguiente tabla:

Tipo de actividad	Calorías quemadas [cal]
REPOSO (si la actividad se realiza a menos de 0.5 km/h)	0
ACTIVIDAD MODERADA (si la actividad se realiza a más de 0.5 km/h y menos de 7 km/h)	$0.0029 * (pesodelusuario[kg] * 2.2) * tiempo[mi$
ACTIVIDAD INTENSA (si la actividad se realiza a 7 km/h o más)	$0.0048 * (pesodelusuario[kg] * 2.2) * tiempo[mi$

Escribir un programa que implemente el algoritmo de dicha aplicación. El programa debe permitir ingresar el peso del usuario y calcular las calorías quemadas durante el día. Para simular la señal de entrada del GPS, generar un vector (de longitud N) con los datos del desplazamiento (en m) del usuario, almacenando números aleatorios de tipo real, entre 0 y 200 (suponer que se toman muestras cada 30 segundos). Además, el programa debe calcular y mostrar el porcentaje de tiempo que el usuario estuvo realizando cada tipo de actividad.

9) Considerar la siguiente tabla de valores de una función *f*:

x	3.0	3.3	3.6	4.0	4.5
f(x)	14.9	18.2	22.3	27.2	33.3

Se desea usar estos valores para estimar los valores de f para otros valores de x dentro del intervalo dado por la tabla. Escribir un programa que almacene la tabla anterior y que para cada valor de x elegido por el usuario, escriba la correspondiente aproximación de *f(x)*.

Nota: Esta aproximación puede ser obtenida mediante el método de *interpolación lineal*. Este consiste en trazar una línea recta entre los dos puntos de la tabla que rodean al valor de x a evaluar, y luego evaluar x en dicha recta. Es decir, si quisiéramos aproximar el valor $f(3.5)$, los dos puntos serían los correspondientes a $x = 3.6$ y $x = 3.3$

10) Desarollar un procedimiento *sinExtremos*, que reciba un vector de números y lo devuelva sin el máximo ni el mínimo. Por ejemplo: si se recibe el vector [38, 3, 86, 341, 29, 42, 35, 9] se deberá devolver [38, 86, 29, 42, 35, 9].
Nota: si el vector tiene menos de dos elementos, *sinExtremos* debe devolver el mismo vector y un cartel de advertencia.

11) Ingresar N enteros en un arreglo A y luego:

a) Calcular la máxima diferencia en valor absoluto entre dos componentes consecutivas.
b) Crear dos vectores B y C de modo tal que, B tenga sólo las componentes de A que ocupan lugares pares y C sólo las componentes de A que ocupan lugares impares. Mostrar los dos vectores.
c) Calcular la suma de los valores negativos (sin el mínimo) y el producto de los valores positivos (excluir el máximo) no nulos del vector A.
d) Hallar la media y la varianza de los datos, siendo:

$$varianza: \frac{\sum_{i=1}^{n} (x_i - \acute{x})^2}{n}$$

$$media: \frac{\sum_{i=1}^{n} x_i}{n}$$

e) Ordenar el arreglo A en forma ascendente y obtener la mediana. Recordar que la mediana es el elemento que se encuentra en la posición central, si N es impar, o es el promedio de los dos elementos centrales, si N es par.

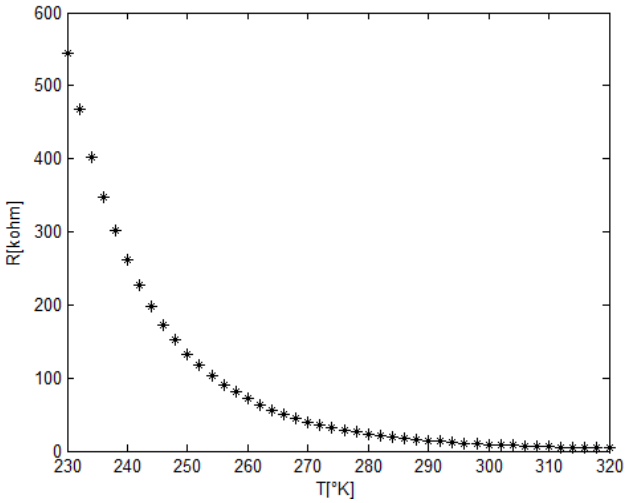
12) Una roca de masa m se cuelga de un resorte y se arrastra hacia abajo una longitud A. Al soltarla, la roca oscilará hacia arriba y hacia abajo con una posición vertical en función del tiempo dada por:

$$y(t) = A \,.\, e^{-\gamma t} \cos\left(\sqrt{\frac{k}{m}} \,.\, t\right)$$

donde, $y = 0$ corresponde a la posición vertical (de reposo) de la roca cuando simplemente se cuelga del resorte.
Siendo $k = 4$ kg/s^2 , $\gamma = 0.15$ s^{-1}, asumiendo que la roca tiene una masa $m = 9$kg, y que se arrastra la roca hacia abajo una longitud $A = 0.3$ m:

a) Almacenar en dos arreglos t_array e y_array de tamaño 101, tiempos y posiciones de la roca, para un rango [0 s, 25 s], con tiempos igualmente espaciados.
b) Indicar el primer intervalo de tiempo durante el cual el estiramiento del resorte no supera la mitad de la amplitud máxima A.

13) Un dispositivo sensor de temperatura opera variando la resistencia que ofrece a la circulación de corriente, en función de la temperatura. La relación entre resistencia-temperatura es completamente alineal y se representa mediante la siguiente curva:



Los datos de la curva se encuentran en °K en un archivo Temperatura.txt (provisto por la cátedra en el Campus Virtual) y se almacenan de la siguiente forma: en la primera línea figura la cantidad de datos, y a partir de la segunda línea, se disponen las relaciones resistencia-temperatura en cada línea del archivo.
Escribir un programa que almacene en una estructurada adecuada la información de dicha curva y ofrezca un menú que permita:

a) Mostrar las relaciones resistencia-temperatura leídas desde el archivo expresando la temperatura en °C. La relación entre °K y °C es la siguiente: °C= °K - 273.15.
b) Obtener y mostrar en pantalla la temperatura aproximada en °C asociada a un valor de resistencia ingresado por el usuario. Para obtener la temperatura aproximada, utilice el valor de la tabla que más se aproxime al valor ingresado.
c) Para una cantidad N de valores de resistencia ingresados por el usuario, generar y mostrar un arreglo con las temperaturas asociadas en °C. Utilice el criterio de proximidad empleado en el inciso anterior.

14) Se tiene un vector V de N componentes reales ordenadas en forma ascendente, y dos valores reales A y B (A<B).
Se pide crear, de forma eficiente, un vector W con los elementos positivos de V que se encuentren en el intervalo [A, B]. Mostrar W o mostrar un cartel aclaratorio si no se ha podido generar.

15) La regla de Paolo Ruffini (1765-1822) sirve para realizar de forma sencilla la división de un polinomio (de grado mayor que uno) por un binomio de la forma x-a. Ejemplo:

La división de x^3+2x^2-x-2 por $x-3$ se realiza como se muestra

	1	2	-1	-2
3		3	15	42
	1	5	14	40

El cociente de la división es $x^2+5x+14$ y el resto 40. Además, habiendo tomado como divisor a $x-3$, dicho resto 40 es también la evaluación del polinomio en el valor 3.

Se desea:

a) Realizar un programa que aplique la regla de Ruffini para dividir un polinomio $p(x)$ de coeficientes enteros por un binomio de la forma $x-a$, siendo a un entero.
b) Realizar un programa que calcule cuántas veces es divisible un polinomio de coeficientes enteros por el binomio $x-a$, donde a es un entero.
c) Realizar un programa que calcule todas las raíces enteras de un polinomio, sabiendo que cualquier raíz entera debe ser un divisor del término independiente.

16) Analizar la prueba de escritorio, para el programa trascripto a continuación, e indicar con qué formato aparece la salida en pantalla.

```
PROGRAM EJEM5;
TYPE
    Matriz=Array[1..5,1..5] of Real;
VAR
    M,T: Matriz;
    F,C,i,j:Byte;

Begin
    Writeln('Ingrese la cantidad de filas');
    Readln(F);
    Writeln('Ingrese la cantidad de columnas');
    Readln(C);
    For i:=1 to F Do
        For j:=1 to C Do
              Begin
                Writeln('Ingrese el valor de la componente ',i,',',j);
                Readln(M[i,j]);
              End;
    For i:=1 to F Do
        For j:=1 to C Do
            T[i,j]:=M[i,j]+i+j;
```

```
        For i:=1 to F Do
            Begin
                For j:=1 to C Do
                    Write(T[i, j]);
                Writeln;
            End;
    End.
```

F	C	i	j	M	T	Salida
2	3					
		1	1	[-2,...] ...		
			2	[-2,4,...] ...		
			3	[-2,4,0,...] ...		
		2	1	[-2,4,0,...] [-8,...] ...		
			2	[-2,4,0,...] [-8,7,...] ...		
			3	[-2,4,0,...] [-8,7,5,...] ...		
		1	1		[0,...] ...	
			2		[0,7,...] ...	
			3		[0,7,4,...] ...	
		2	1		[0,7,4,...] [-5,...] ...	
			2		[0,7,4,...] [-5,11,...] ...	
			3		[0,7,4,...] [-5,11,10,...] …	
		1	1,2,3			0 7 4
		2	1,2,3			-5 11 10

17) Escribir un programa modularizado que permita leer los FxC elementos reales de una matriz M y luego:

a) Asignar a todos los elementos de la cuarta fila el valor cero.
b) Asignar a todos los elementos de la columna H (H elegida por el usuario) el valor 25.
c) Multiplicar por -1 a todos los elementos con índice de fila menor que índice de columna.
d) Permutar las columnas S y P (elegidas por el usuario). Verificar que S y P sean menores o iguales que C y distintas entre sí.
e) Mostrar la matriz modificada con formato matricial.

18) Desarrollar un programa que implemente procedimientos y/o funciones que permitan:

a) Leer una variable A de tipo MATRIZ de FxC números enteros
b) Dado un número h de columna ($1 \leq h \leq C$), calcular el promedio de los elementos de dicha columna.
c) Dado un número k de fila ($1 \leq k \leq F$), calcular el mínimo de los elementos de dicha fila.
d) Obtener un vector de C elementos con los promedios de cada columna de la matriz A
e) Obtener un vector de F elementos con los mínimos de cada fila de la matriz A.
f) Generar un vector V con los elementos de A que sean positivos.

19) Desarrollar funciones o procedimientos que cumplan lo pedido en cada inciso:

a) *filaMati*: devuelve una lista con la fila i-ésima de la matriz.
b) *columnaMatj*: devuelve una lista con la columna j-ésima de la matriz.
c) *diagonalPrincipalMat*: devuelve una lista con los valores de la diagonal principal de la matriz.
d) *parteSuperiorMat* y *parteInferiorMat*: devuelven listas con los valores por encima y por debajo de la diagonal principal de la matriz.

e) *esIdentidadMat*: debe indicar si una matriz es o no identidad.

Por ejemplo, para:

$$M=\begin{pmatrix}1 & 2 & 3\\ 2 & 3 & 4\\ 7 & 8 & 9\end{pmatrix},$$

filaMati(M,1) devuelve (1, 2, 3)
columnaMatj(M,3) devuelve (3, 4, 9)
diagonalPrincipalMat(M) devuelve (1, 3, 9)
parteSuperiorMat(M) devuelve (2, 3, 4)
parteInferiorMat(M) devuelve (2, 7, 8)
esIdentidadMat(M) devuelve FALSE
En el caso de *parteSuperior* y *parteInferior* no importa el orden.

20) Defina un procedimiento para descomponer una matriz cuadrada M en otras dos, A y S, con sus mismas dimensiones, de manera que

$$M = S + A$$

y tales que S es simétrica y A antisimétrica. Esto se consigue forzando que

$$S_{ij}=S_{ji}=\frac{M_{ij}+M_{ji}}{2}, \forall i, j \in \{1,\ldots,N\}$$

y

$$A_{ij}=-A_{ji}=\frac{M_{ij}-M_{ji}}{2}, \forall i, j \in \{1,\ldots,N\}$$

<u>Ejemplo</u>:

$$M=\begin{pmatrix}2 & 1 & 8\\ 5 & -1 & 0\\ 9 & 3 & 1\end{pmatrix}, S=\begin{pmatrix}2 & 3 & 17/2\\ 3 & -1 & 3/2\\ 17/2 & 3/2 & 1\end{pmatrix} y\ A=\begin{pmatrix}0 & -2 & -1/2\\ 2 & 0 & -3/2\\ 1/2 & 3/2 & 0\end{pmatrix}$$

21) Dadas las siguientes definiciones:

```
Type
TMatriz = array[1.. MAX, 1.. MAX] of integer;
```

donde MAX es una constante definida anteriormente. Se pide escribir, de forma eficiente, la función:

```
Function SumaDiagonalesIguales(matriz : TMatriz) : boolean;
```

que devuelve *true* si las sumas de las distintas diagonales descendentes son todas iguales. Por ejemplo, si tenemos la matriz:

$$\begin{pmatrix}A & B & C & D\\ E & F & G & H\\ I & J & K & L\\ M & N & O & P\end{pmatrix}$$

todas sus diagonales descendentes son:

```
M
I   N
E   J   O
A   F   K   P
B   G   L
C   H
D
```

22) En un juego de tiro al blanco se han realizado una serie de N disparos, cada uno representado como coordenadas (x,y)

Se pide escribir un programa Pascal para:

- Leer los disparos, almacenándolos en una matriz de Nx2 elementos.
- Sabiendo que un punto de coordenadas (X, Y), ingresadas por teclado, es el centro del blanco, mostrar alguno de los siguientes resultados, elegido por el operador a partir de un menú de opciones.
  a) El porcentaje de tiros que han caído dentro de un círculo de radio R (dato leído por teclado) y centro (X, Y), respecto del total de disparos registrados.
  b) Número de orden del primer disparo que dio en el centro, si es que hubo alguno.
  c) Las coordenadas de los puntos que han quedado más lejos del centro (considere que puede haber más de uno).

  El menú debe reiterarse hasta que el usuario decida salir.

23) La distancia que hay entre N ciudades se almacena en una matriz DISTANCIA donde DISTANCIA[i,j] indica la distancia en km que hay entre la ciudad i y la ciudad j.

Se pide ingresar la información necesaria para armar dicha matriz, almacenar los nombres de las ciudades en una estructura auxiliar, y luego:

a) Calcular e informar: el par de ciudades más distantes entre sí, y el par más cercanas.
b) Para una ciudad X ingresada por teclado, informar qué tan distante está de las restantes ciudades.

EJEMPLO: Para N=5

y las ciudades:

1. Ciudad de Bs As
2. Córdoba
3. Corrientes
4. Formosa
5. La Plata

	1	2	3	4	5
1	0				
2	646	0			
3	792	677	0		
4	933	824	157	0	
5	53	698	830	968	0

La Plata y Formosa son las ciudades más distantes.

Ciudad de Bs As y La Plata son las ciudades más cercanas.

Para X=Corrientes

Ciudad de Bs As a 792 km
Córdoba a 677 km
Formosa a 157 km
La Plata a 830 km

24) Un sistema de energía renovable funciona mediante un conjunto de N aerogeneradores. Cada 24 h, durante M días, se registra la cantidad de vueltas de cada aerogenerador. Las aspas tienen una longitud de 10 m de radio.

Escribir un programa modularizado que permita calcular:

a) La velocidad angular promedio de cada aerogenerador, para cada día, en rad/seg.
b) La aceleración centrípeta en el centro del aspa, del aerogenerador que obtuvo la mayor velocidad promedio.
c) El día de menor viento.

25) El Servicio Meteorológico Nacional registra, mes por mes, la cantidad de lluvia que cae por provincia, en mm (entero), y lo almacena en una matriz. La información se puede ingresar de forma desordenada. Los datos que se ingresan son MES, PROVINCIA y MM.

Nota: Codificar los nombres de las provincias con un entero del 1 al 23

Realizar un programa que permita:

a) Ingresar, desde un archivo, N datos de la matriz (correspondientes a los meses en que se produjeron lluvias). El nombre de la provincia se recibe codificado.
b) Mostrar la matriz de datos.
c) Calcular en qué mes y en qué provincia llovió más. Mostrar MES, PROVINCIA (el nombre), y cuántos MM llovieron.
d) Hacer un listado de las provincias con su precipitación anual, indicando el nombre de la provincia y la cantidad de agua caída.